

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

Branches e merging

Aula 1

Código da aula: [DADOS]ANO1C1B4S27A1

Controle de versão
com Git e GitHub

Mapa da Unidade 8 Componente 1

semana

25

Branches no Git

Você está aqui!

Branches e merging

semana

27

semana

20

Fundamentos
de Git

semana

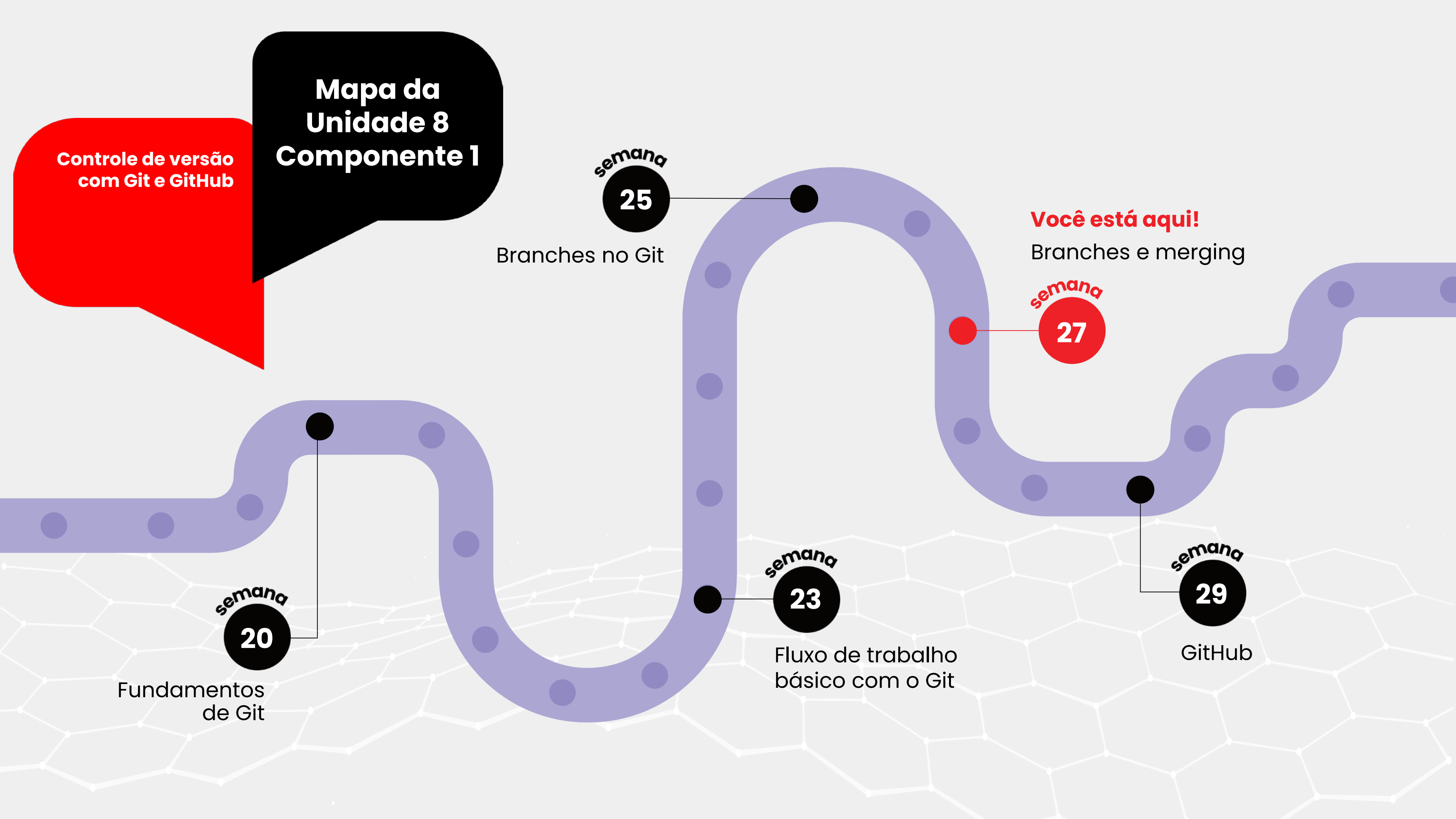
23

Fluxo de trabalho
básico com o Git

semana

29

GitHub



**Controle de versão
com Git e GitHub**

Mapa da Unidade 8 Componente 1

Você está aqui!

Branches e merging

Aula 1

Código da aula: [DADOS]ANO1C1B4S27A1

27



Objetivos da Aula

- Introduzir e utilizar ferramentas de desenvolvimento de software como Git e GitHub.



Recursos Didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou à internet.



Duração da Aula

50 minutos.



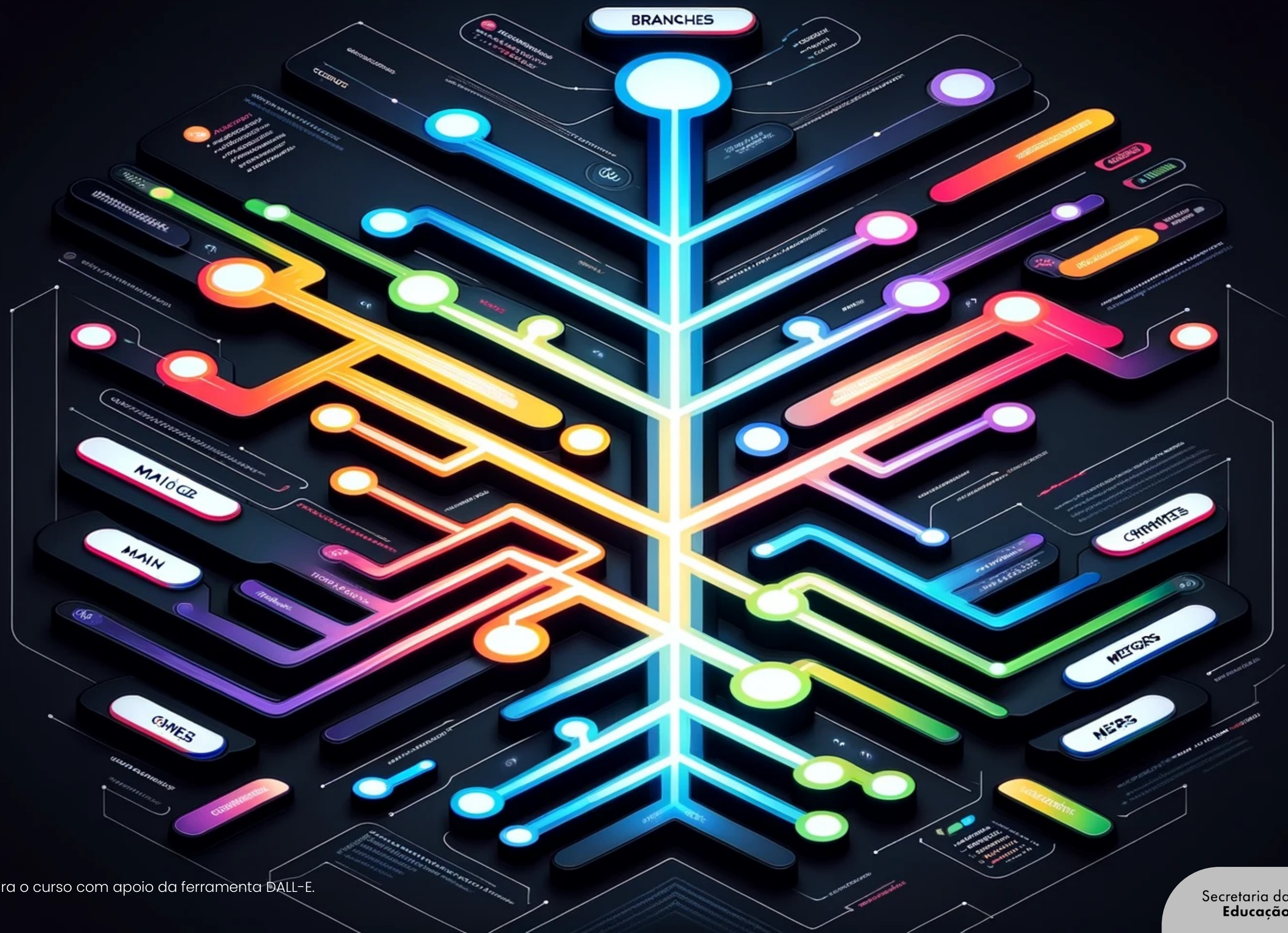
Competências Técnicas

- Usar ferramentas de desenvolvimento de software como Git e GitHub;
- Compreender e dominar técnicas de manipulação de dados.



Competências Socioemocionais

- Analisar informações de forma crítica e tomar decisões; avaliar a qualidade dos dados e a eficácia dos modelos e das técnicas de análise.
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados; trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes.



Primeiras ideias

Com base na imagem, responda:

-  O que você acha que são branches em um sistema de controle de versão como o Git?
-  Por que é importante utilizar branches quando se trabalha em um projeto colaborativo?
-  O que é um merge e qual é seu papel no fluxo de trabalho com branches?

Ponto de partida

Imagine que você e sua equipe estão trabalhando em um aplicativo de rede social.

Cada membro da equipe está responsável por diferentes funcionalidades: um está trabalhando no feed de notícias, outro está implementando o sistema de mensagens e um terceiro está corrigindo um bug crítico na autenticação de usuários.

Todos vocês estão usando Git para versionamento do código.

Discussão:

1. Como vocês organizariam o trabalho em branches para garantir que cada desenvolvedor possa trabalhar de forma independente?
2. Que desafios vocês podem enfrentar ao fazer merge das branches de diferentes desenvolvedores?

Situação fictícia elaborada especialmente para o curso.

Construindo o **conceito**

O que são branches?

Branches em Git **são ramificações do histórico de commits do projeto**, permitindo que desenvolvedores trabalhem de forma isolada em diferentes funcionalidades ou correções sem afetar a branch principal.

Quando um desenvolvedor **cria uma branch, ele está criando um novo contexto de trabalho derivado de um commit específico**. Isso é especialmente útil para desenvolver novas funcionalidades, corrigir bugs ou experimentar ideias sem comprometer o estado do código na branch principal.

Tipos de branches

Existem vários tipos de branches que podem ser usadas conforme o propósito do desenvolvimento:

- ▶ **Main:** a branch principal, que contém o código pronto para produção.
- ▶ **Feature:** criadas para desenvolver novas funcionalidades ou melhorias específicas.
- ▶ **Release:** utilizadas para preparar uma nova versão de lançamento do software.
- ▶ **Hotfix:** criadas para corrigir rapidamente bugs críticos encontrados em produção.

Construindo o **conceito**

Criando e gerenciando branches

Comandos básicos:

- ▶ **Criar uma branch:** `git branch <nome-da-branch>`
- ▶ **Mudar para uma branch:** `git checkout <nome-da-branch>` ou `git switch <nome-da-branch>`
- ▶ **Criar e mudar para uma branch:** `git checkout -b <nome-da-branch>` ou `git switch -c <nome-da-branch>`
- ▶ **Listar branches:** `git branch`
- ▶ **Deletar uma branch:** `git branch -d <nome-da-branch>` ou `git branch -D <nome-da-branch>`

Construindo
o **conceito**

Práticas recomendadas

- ▶ Usar nomes descritivos que refletem o propósito da branch.
Exemplo: feature/login-page, hotfix/security-issue.
- ▶ Seguir convenções de nomenclatura consistentes dentro da equipe.

Construindo
o **conceito**

O que é merge

Merge é a operação **de integração das mudanças feitas em uma branch de volta para outra branch**, geralmente a principal. Isso combina os históricos de commits das duas branches, incorporando as alterações de uma na outra.

O processo de merge combina os históricos de commits das duas branches.

Construindo
o **conceito**

Tipos de merges

- ▶ **Fast-forward:** o merge mais simples, onde o ponteiro da branch principal é movido para o commit da branch de feature, desde que não haja commits novos na branch principal.
- ▶ **Three-way merge:** utilizado quando houver novos commits na branch principal desde que a branch de feature foi criada. Git calcula um commit de merge combinando as mudanças das duas branches.

Construindo
o **conceito**

Conflitos de merge

Conflitos de merge ocorrem quando o Git não consegue reconciliar automaticamente as diferenças entre duas branches.

Isso geralmente acontece quando mudanças conflitantes foram feitas nos mesmos trechos de código.

Construindo
o **conceito**

Conflitos de merge – Resolução

Como resolver conflitos de merge:

- ▶ **Identificar os conflitos:** Git marca os conflitos no código com indicadores (<<<<<, =====, >>>>>).
- ▶ **Resolver manualmente:** editar os arquivos para reconciliar as diferenças.
- ▶ **Marcar como resolvido:** após resolver os conflitos, usar **git add <arquivo>** e **git commit** para concluir o merge.

Construindo
o **conceito**

Fluxos de trabalho comuns

Git Flow

Um modelo de branching que define branches específicas para desenvolvimento, releases e correções rápidas. Inclui as branches principais: **main, develop, feature, release e hotfix.**

GitHub Flow

Um fluxo mais simples, adequado para o desenvolvimento contínuo. Utiliza branches curtas e pequenas PR (pull requests) para integrar mudanças na branch **main**.

GitLab Flow

Combina elementos do Git Flow e do GitHub Flow, incorporando controle de ambiente e deploy contínuo. Enfatiza a entrega contínua e a integração de branches de desenvolvimento com ambientes específicos.

Construindo o **conceito**

Exemplos

Vamos praticar: adicionando uma função de soma e revertendo um commit

1. Inicializar um repositório Git:

```
!mkdir simple_git_project  
%cd simple_git_project  
!git init
```

2. Criar um arquivo Python com uma função simples:

```
%%writefile sum.py  
def add_numbers(a, b):  
    return a + b  
  
print(add_numbers(2, 3))
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplos

3. Commitar mudanças:

```
!git add sum.py  
!git commit -m "Add sum function"
```

4. Criar uma nova branch e fazer mudanças:

```
!git checkout -b feature/addition-feature
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplos

5. Modificar o arquivo Python na nova branch:

```
%%writefile sum.py
def add_numbers(a, b):
    return a + b

def subtract_numbers(a, b):
    return a - b

print(add_numbers(2, 3))
print(subtract_numbers(5, 2))
```

6. Commitar mudanças na nova branch:

```
!git add sum.py
!git commit -m "Add subtract function"
```

Construindo
o **conceito**

Exemplos

7. Voltar para a branch principal (main) e mesclar a nova branch:

```
!git checkout main  
!git merge feature/addition-feature
```

8. Reverter o último commit:

```
!git revert HEAD
```

Você também pode usar:

```
!git reset --soft HEAD~1
```

```
!git reset --hard HEAD~1
```

Elaborado especialmente para o curso com o prompt de comando.



Vamos
fazer um
quiz

Qual comando cria uma branch?

``git init``

``git branch``

``git commit``

``git merge``



Vamos
fazer um
quiz

**Qual comando alterna entre
branches?**

``git checkout``

``git clone``

``git push``

``git pull``



Vamos
fazer um
quiz

O que é um merge fast-forward?

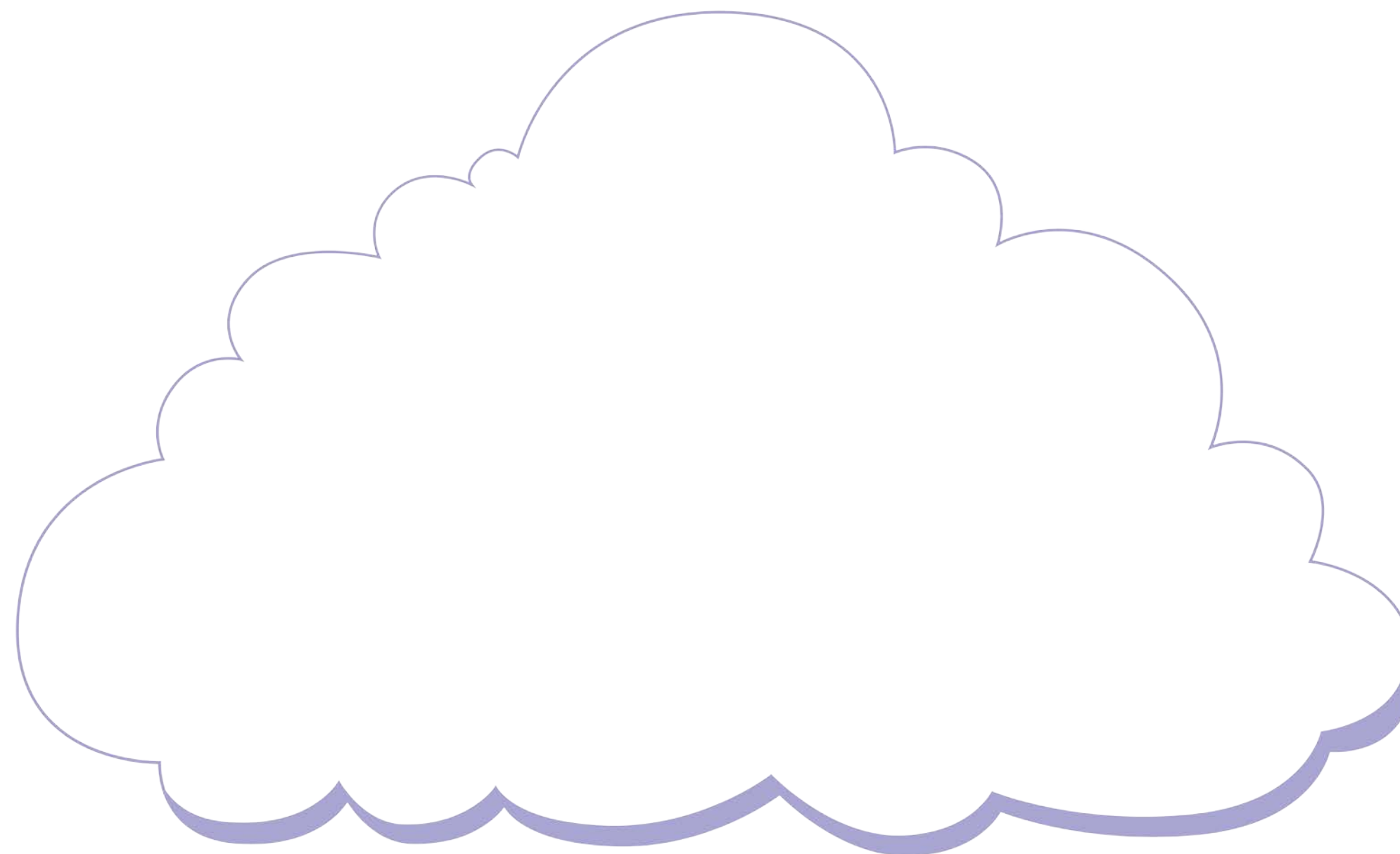
Move pointer.

Combina changes.

Detecta conflitos.

Inicia repositório.

Nuvem de palavras



© Getty Images

O que nós
**aprendemos
hoje?**



© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** Branches em Git permitem que desenvolvedores trabalhem em diferentes funcionalidades de forma isolada, sem afetar a branch principal. Existem vários tipos de branches, como main, feature, release e hotfix, cada uma com um propósito específico.
- 2** Criar e gerenciar branches é simples, com comandos como git branch, git checkout, e git switch. Merges combinam mudanças de uma branch para outra, com tipos como fast-forward e three-way merge. Conflitos de merge ocorrem quando há mudanças conflitantes.
- 3** Fluxos de trabalho como Git Flow, GitHub Flow e GitLab Flow ajudam a organizar o desenvolvimento. Cada um deles tem suas próprias práticas e convenções para gerenciar branches e integrações de maneira eficiente.

Saiba mais

É hora de se tornar tão forte quanto um ninja em Git!

Acesse o material complementar e veja como aplicar conhecimentos em GitFlow e GitHub Actions!

ALURA. *DevOps: trabalhando com Repositórios no GitHub*.

Disponível em: <https://www.alura.com.br/curso-online-devops-trabalhando-repositorios-github>.

Acesso em: 23 jul. 2024.

Referências da aula

BELL, P.; BEER, B. *Introdução ao GitHub*: um guia que não é técnico. São Paulo: Novatec, 2015.

GITHUB DOCS. *Documentação de introdução ao GitHub*, [s.d.]. Disponível em: <https://docs.github.com/pt/get-started>. Acesso em: 23 jul. 2024.

Identidade visual: imagens © Getty Images

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**